dependent data is business critical, such as orders, this is not desirable. You would clearly want to retain historical data about bookings at a property, despite the fact that it has since been bulldozed, for example. You may even need to restore it if it has been deleted accidentally. For this reason, you will more often employ a *soft delete*, whereby a record is flagged in some way as deleted, rather than removing the record entirely, which is what the `DbContext.Remove` method results in. In the final section of this chapter, you will add a new property to the `Property` class representing the date and time an entity was marked as deleted. You will add a new migration to update the database schema, and then modify the Delete pages to accommodate your revised strategy for managing deletes.

The first step is to add a nullable `DateTime` property to the `Property` class:

```
public DateTime? Deleted { get; set; }
```

I usually use a `DateTime`, rather than a `bool`, for this purpose because someone will inevitably ask *when* the item was deleted. In the absence of more sophisticated logging, at least this can help me answer that question.

I'll take this opportunity to reinforce the point about overposting attacks. You have just added a new property to your class that you do not want users to set directly. If you allow the model binder to bind directly to an instance of the class, you will expose this property to users. By specifying only individual properties as binding targets, you prevent the user from being able to set the `Deleted` property's value.

Having added the `Deleted` property, you can add a new migration using either the package manager console or the command line, which will detect the changes you have made to the model and will reflect those changes by